PRINCETON UNIVERSITY PRESS

Chapter Title: ITERATED GROWTH

Book Title: Natural Complexity

Book Subtitle: A Modeling Handbook

Book Author(s): Paul Charbonneau

Published by: Princeton University Press

Stable URL: https://www.jstor.org/stable/j.ctt1vwmgzt.5

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

Princeton University Press is collaborating with JSTOR to digitize, preserve and extend access to *Natural Complexity*

JSTOR

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

2

ITERATED GROWTH

The bewildering array of complex shapes and forms encountered in the natural world, from tiny crystals to living organisms, often results from a growth process driven by the repeated action of simple "rules." In this chapter we examine this general idea in the specific context of *cellular automata* (hereafter, often abbreviated as CAs), which are arguably the simplest type of computer programs conceivable. Yet they can sometimes exhibit behaviors that, by any standard, can only be described as extremely complex.

Cellular automata also exemplify, in a straightforward computational context, a recurring theme that runs through all instances of natural complexity to be encountered in this book: simple rules can produce complex global "patterns" that cannot be inferred or predicted even when complete, a priori knowledge of these rules is at hand.

2.1 Cellular Automata in One Spatial Dimension

Imagine a one-dimensional array of contiguous cells, sequentially numbered by an index j starting at $j = 0$ for the leftmost cell. Each cell can be "painted" either white or black, with the rule for updating the jth cell depending only on its current color and those of the two neighboring cells at positions $j - 1$ and $j + 1$. Consider now the following graphical procedure: At each

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

iteration, the CA looks like a linear array of cells that are either black or white. If we now stack successive snapshots of this row of cells below one another, we obtain a two-dimensional spatiotemporal picture of the CA's evolution, in the form of a (black and white) pixelated image such as formed on the CCD of a digital camera, except that each successive row of pixels captures an iteration of the growth process, rather than the vertical dimension of a truly 2-D image.

The simple question then is, starting from some given initial pattern of white and black cells, how will the array evolve in response to the repeated application of the update rule to every cell of the array? As a first example, consider the following very simple CA update rule:

- **First rule.** Cell j becomes (or stays) black if one or more of the neighbor triad $[j-1, j, j+1]$ is black; otherwise it becomes (or stays) white.

The top panel of figure 2.1 shows the first 20 iterations of a CA abiding by this rule, starting from a single black cell in the middle of an otherwise all-white array. On this spatiotemporal diagram, the sideways growth of the CA translates into a black triangular shape expanding by one cell per iteration from the single initial black cell. Now start again from a single black cell but adopt instead the following, equally simple, second update rule:

- **Second rule.** Cell j becomes (or stays) black if either or both of its neighbors $j-1$ and $j+1$ are black; otherwise it stays (or becomes) white.

This yields the pattern plotted in the bottom panel of figure 2.1. The global shape is triangular again, but now the interior is a checkerboard pattern of white and black cells alternating regularly in both the spatial and temporal dimensions. With just a bit of thinking, these two patterns could certainly have been expected on the basis of the above two rules.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

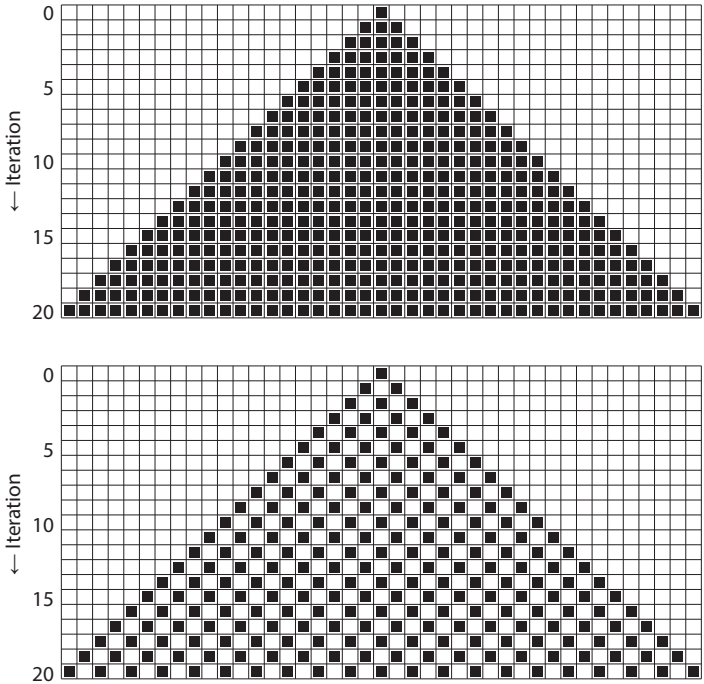

Figure 2.1. The first 20 iterations of a 1-D CA abiding by the first (top) and second (bottom) update rules introduced in the text, in both cases starting from a single black cell at the center of the array (iteration 0, on top). The horizontal direction is the "spatial" dimension of the 1-D CA, and time (iterations) runs downward.

But is it always the case that simple CA rules lead to such simple, predictable spatiotemporal patterns? Consider now the following update rule:

- **Third rule.** Cell j becomes (or stays) black if one and only one of its two neighbors $j-1$ or $j+1$ is black; otherwise it stays (or becomes) white.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

This is again a pretty simple update rule, certainly as simple as our first and second rules. The top panel of figure 2.2 shows the pattern resulting from the application of this rule to the same initial condition as before, namely, a single black cell at the array center. The globally triangular shape of the structure is again there, but the pattern materializing within the structure is no longer so simple. Many white cells remain, clustered in inverted triangles of varying sizes but organized in an ordered fashion. This occurs because our third rule implies that once the cells have reached an alternating pattern of white/black across the full width of the growing structure, as on iterations 3, 7, and 15 here, the rule forces the CA to revert to all white at the next iteration, leaving only two black cells at its right and left extremities. Right/left symmetrical growth of new black cells then resumes from these points, replicating at each end the triangular fanning pattern produced initially from the single original black cell. Growth thus proceeds as a sequence of successive branching, fanning out, and extinction in the interior.

The bottom panel of figure 2.2 displays 512 iterations of the same CA as in the top panel, with the cell boundaries now omitted. Comparing the top and bottom panels highlights the fundamental difference between the "microscopic" and "macroscopic" views of the structure. On the scale at which the CA is operating, namely, the triad of neighboring cells, a cell is either white or black. On this microscopic scale the more conspicuous pattern to be noticed in the top panel of figure 2.2 is that blacks cells always have white cells for neighbors on the right, left, above, and below, but no so such "checkerboard" constraint applies to white cells (unlike in the bottom panel of figure 2.1). At the macroscopic level, on the other hand, the immediate perception is one of recursively nested white triangles. In fact, the macroscopic triangular structure can be considered as being made from three scaled-down copies of itself, touching at their vertices; each of these three copies, in turn, is made up of three scaled-down

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

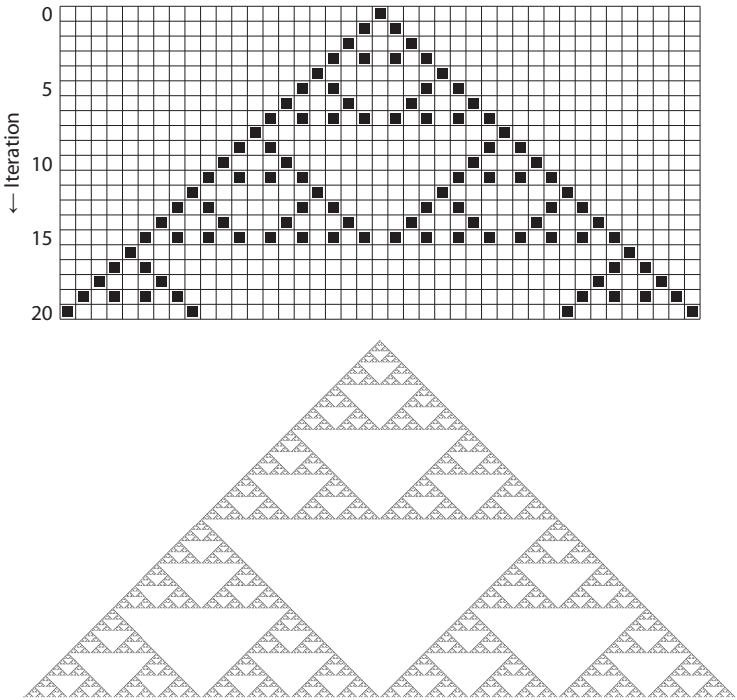

Figure 2.2. The top panel is identical in format to figure 2.1, but now shows the structure produced after 20 iterations by the third CA update rule, introduced in the text. The bottom panel shows the same CA, now pushed to 512 iterations, with cell boundaries removed for clarity.

copies of itself, and so on down to the "microscopic" scale of the individual cells.[1]

[1]This structure belongs to a class of geometrical objects known as *Sierpinski triangles*. It can be constructed by a number of alternative geometrical procedures. The simplest consists in drawing a first triangle, then partitioning it into four smaller triangles by tracing three straight-line segments joining the edge centers, then repeating this process for the three outer triangles so produced, and so on. The CA, in contrast, generates the same macroscopic structure via a directional iterative spatiotemporal growth process.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

This type of recursive nesting is a hallmark of *self-similarity*, and flags the structure as a *fractal*. For now you may just think of this concept as capturing the fact that successive zooms on a small part of a structure always reveal the same geometrical pattern, like with the bifurcation diagram for the logistic map encountered in chapter 1 (see figure 1.2). More generally, self-similarity is a characteristic feature of many complex systems, and will be encountered again and again throughout this book.

Consider, finally, a fourth CA update rule, hardly more complicated than our third:

- **Fourth rule.** Cell j becomes (or stays) black if one and only one of the triad $[j-1, j, j+1]$ is black, or if only j and $j+1$ are black; otherwise it stays (or becomes) white.

This rule differs from the previous three in that it now embodies a directional bias, being asymmetrical with respect to the central cell j: a white cell at $j-1$ and black cells at j and $j+1$ will leave j black at the next iteration, but the mirror configuration, $j-1$ and j black and $j+1$ white, will turn j white. The top panel of figure 2.3 shows the first 20 iterations of this CA, as usual (by now) starting from a single black cell. The expected symmetrical triangular shape is there again, but now the interior pattern lacks mirror symmetry, not surprisingly so perhaps, considering that our fourth rule itself lacks right/left symmetry. But there is more to it than that. Upon closer examination one also realizes that the left side of the structure shows some regularities, whereas the right half appears not to. This impression is spectacularly confirmed upon pushing the CA to a much larger number of iterations (bottom panel). On the right the pattern appears globally random. As with our third rule, inverted white triangles of varying sizes are generated in the course of the evolution, but their spatial distribution is quite irregular and does not abide by any obvious recursive nesting pattern. On the left,

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

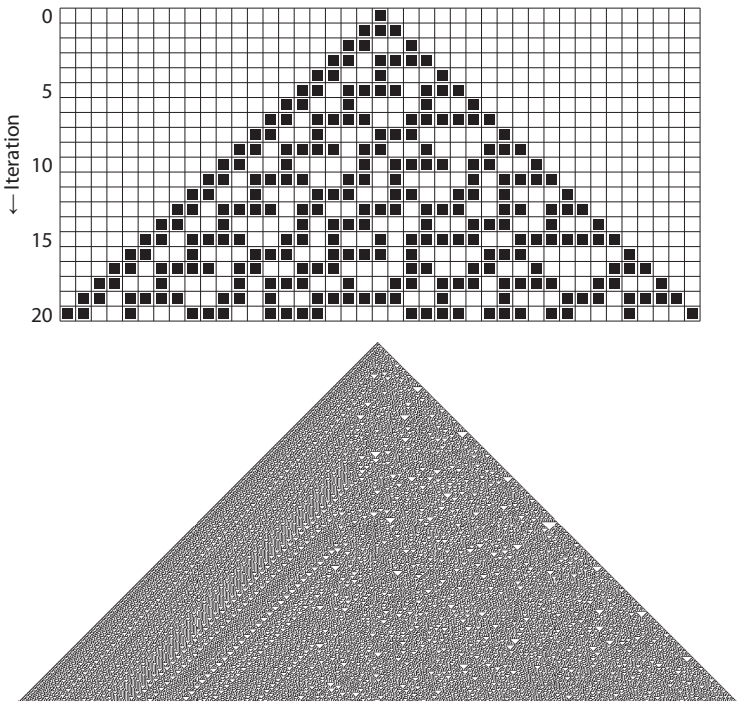

Figure 2.3. Identical in format to figure 2.2, but now for the fourth CA update rule introduced in the text.

in contrast, the pattern is far more regular, with well-defined structures of varying periodicities recurring along diagonal lines running parallel to the left boundary of the structure.

With eight possible three-cell permutations of two possible states (white/black, or 0/1, or whatever) and evolution rules based on three contiguous cells (the cell itself plus its right and left neighbors), there exist 256 possible distinct evolutionary rules.[2]

[2]Describing CA rules in words, as we have done so far, can rapidly become quite awkward. A much superior and compact description can be made using an 8-bit binary string, with each bit giving the update (black = 0 and white = 1,

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

Even if always starting from a single active cell, as in figure 2.1, these various rules lead to a staggering array of patterns, going from triangular wedges, repeating checkerboard or stripe patterns, simple or not-so-simple patterns propagating at various angles, nested patterns (as in figure 2.2), mixtures of order and disorder (as in figure 2.3), all the way to complete randomness. You get to explore some of these in one of the computational exercises proposed at the end of this chapter.

These 256 1-D CA rules can be divided fairly unambiguously into four classes, according to the general properties of the end state they lead to, that are independent of the initial condition.[3]

- Class I CAs evolve to a stationary state; our first rule (figure 2.1) offers an example, although keep in mind that the stationary state need not be all black or all white.
- Class II CAs evolve into a periodic configuration, where each cell repeatedly cycles through the same set of states (which may differ from one cell to the next). Our second rule is a particularly simple exemplar of this class.
- Class III CAs evolve into a nonperiodic configuration. Even though figure 2.2 looks quite regular, our third rule belongs,in fact, to this class, as you will get to verify in one of the computational exercises suggested at the end of this chapter.

---

say) associated with one of the eight possible permutations of black/white on three cells. As a bonus, interpreting each such string as the binary coding of an integer yields a number ranging from 0 to 255, which then uniquely labels each possible rule. Chapter 3 of the book by Wolfram, cited in the bibliography, describes this procedure in detail. Under this numbering scheme, the four rules introduced above are numbered 254, 250, 90, and 30, respectively.

[3]The classification is best established through the use of a random initial condition, where every cell in the initial state is randomly assigned white or black with equal probability. In such a situation, it is also necessary to introduce periodic boundary conditions, as if the 1-D CA were in fact defined on a closed ring: the last, rightmost cell acts as the left neighbor of the first, leftmost cell, and vice versa.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

- Class IV CAs collect everything else that does not fit into the first three classes; they are also, in some sense, the more interesting rules in that they yield configurations that are neither stationary, periodic, nor completely aperiodic.

Figure 2.4 gives a minimal source code in the Python programming language for running the 1-D two-state CA of this section, with a value of 0 for white cells and 1 for black cells. The CA evolution is stored in the 2-D array `image`, the first dimension corresponding to time/iteration, and the second to the spatial extent of the CA (line 8). This code uses a single black cell at the lattice center for the initial condition (line 9), and is set up to run the third rule introduced above. The condition "one and only one of the nearest neighbors is black" is evaluated by summing the corresponding numerical values of the cells (line 14); if the sum is 1, then node j turns black at the next iteration (value `1` in array `image`, line 15), and otherwise remains white (initialized value `0`). Periodicity is enforced in the spatial direction (lines 18–19; see appendix D.2 for further detail on this). Upon completion of the CA's evolution over the preset number of iterations, the structure produced is displayed using the `imshow()` function from the matplotlib.pyplot graphics library (lines 22–23).

2.2 Cellular Automata in Two Spatial Dimensions

Cellular automata are readily generalized to two (or more) spatial dimensions, but the various possible lattice geometries open yet another dimension (figuratively speaking!) to the specification of the CA and its update rules. It will prove useful to adopt an alternative, but entirely equivalent, formulation of CAs based on a *lattice* of interconnected *nodes*, conceptually equivalent to the center of cells in figures 2.1–2.3. Figure 2.5 illustrates the idea, for different types of 2-D lattices using different *connectivities* between neighboring nodes.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

```
# 1D 2-STATES CELLULAR AUTOMATON
import numpy as np
import matplotlib.pyplot as plt
#-------------------------------------------------------------------------
N=129                                       # Size of 1D CA
n_iter=64                                   # Number of iterations
#-------------------------------------------------------------------------
image=np.zeros([n_iter,N],dtype='int')      # Initialize lattice to white
image[0,N/2]=1                              # But set central node to black

for iterate in range(1,n_iter):             # Iteration loop

    for j in range(1,N-1):                  # Lattice loop
        if image[iterate-1,j+1]+image[iterate-1,j-1] == 1:  # Third rule
            image[iterate,j]=1              # Turn node black
    # End of lattice loop

    image[iterate,0]=image[iterate,N-2]     # Enforce periodicity
    image[iterate,N-1]=image[iterate,1]
# End of iteration loop

plt.imshow(image,interpolation="nearest")   # Display structure
plt.show()
# END
```

Figure 2.4. A minimal source code in the Python programming language for the 1-D CA introduced in this section. As with all Python codes given in this book, the Python NumPy library is used. The NumPy function `np.zeros()` generates an array of whatever size is given to it as an argument and fills it with zeros; here it generates a 2-D array of size `n_iter`×`N`, through the argument `[n_iter,N]`. The code given here operates according to the third CA rule introduced in the text.

On Cartesian lattices in two spatial dimensions (panels A and B), connectivity typically involves either only the 4 nearest neighbors (in red) on the top/bottom/right/left of a given node (in black), or additionally the 4 neighbors along the two diagonals (panel B).[4] Anisotropic connectivities, as in panel C, can be

[4]The top/down/right/left 4-neighbor connectivity on a Cartesian lattice is sometimes referred to as the *von Neumann neighborhood*, and the 8-neighbor connectivity as the *Moore neighborhood*. See appendix D.1 for more on these matters.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

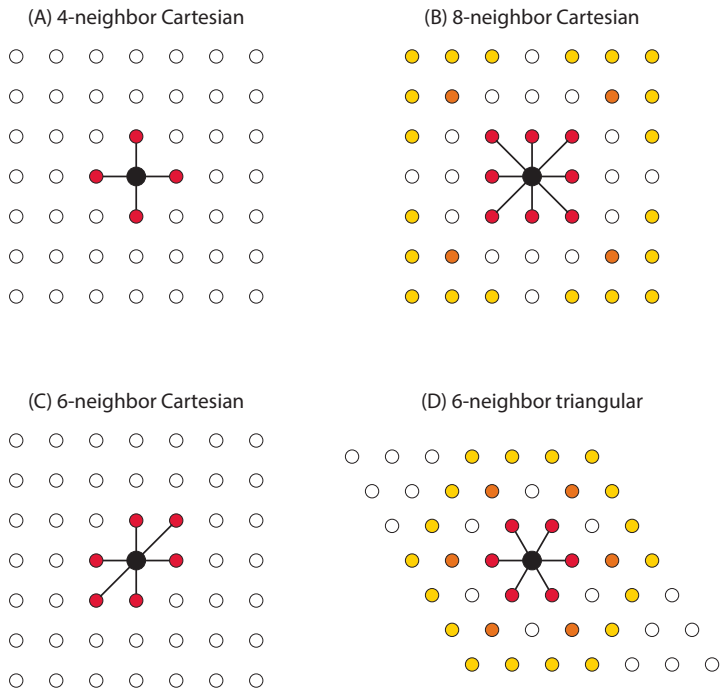

Figure 2.5. Examples of some lattices and associated nodal connectivities, as indicated by the line segments connecting the central black node to its nearest neighbors in red. The meaning of the orange- and yellow-colored nodes in panels B and D will be elucidated in the text.

reinterpreted as changes in lattice geometry. Upon introducing a horizontal displacement of half an internodal distance per row and compressing vertically by a factor $\sin(\pi/3) \simeq 0.866$, as shown in panel D, one obtains a regular triangular lattice with 6-neighbor connectivity. From the point of view of CA evolutionary rules, the two lattices in panels C and D are topologically and operationally equivalent. What is interesting in practice is that whether triangular or Cartesian, these lattices can all be

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

conveniently stored as 2-D arrays in the computer's memory, and the "true" geometry becomes set by the assumed connectivity.

We first restrict ourselves to the following very simple 2-D CA rule:

- A node becomes active if one and only one of its neighboring nodes is also active.

Note that such a rule has no directional bias other than that imposed by the lattice geometry and connectivity: any one active node will do. However, a noteworthy difference from the 1-D CA rules considered previously is that here, once activated a node remains activated throughout the remainder of the iterative process. Nonetheless, as far as rules go, this is probably about as simple as it could get in this context. Figure 2.6 lists a minimalistic Python source code for this automaton, defined on a triangular lattice with 6-neighbor connectivity. Note the following:

1. The code is structured as an outer temporal loop running a preset number of temporal iterations `n_iter` (lines 16–28), inside which two nested loops over each lattice dimension (lines 20–21) carry out the activation test.
2. The connectivity is enforced through the use of the 1-D template arrays `dx` and `dy` in which are hardwired the relative locations, measured in lattice increments, of the connected neighbors (lines 9–10); these 1-D arrays are then used to access the 2-D array `image` which stores the state of the CA proper (lines 23–24).
3. If a cell is to become active at the next iteration, its new state is temporarily stored in the 2-D work array `update` (line 26), which is reset to zero at the beginning of each temporal iteration (line 18); only once all nodes have been tested is the lattice array

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

```
1  # 2D 2-STATES CELULAR AUTOMATON ON TRIANGULAR LATTICE
2  import numpy as np
3  import matplotlib.pyplot as plt
4  #-----------------------------------------------------------------------------
5  N=24                                        # Size of 2D CA
6  n_iter=10                                   # Number of iterations
7  n_neighbor=6                                # Number of connected neighbors
8  #-----------------------------------------------------------------------------
9  dx=np.array([ 1, 0,1,-1,0,-1])              # nearest neighbor template
10 dy=np.array([-1,-1,0, 1,1, 0])
11 image=np.zeros([N,N],dtype='int')           # Initialize lattice to white...
12 image[N//2,N//2]=1                          # ...except central node to black
13 plt.scatter(N//2,N//2)                      # Set up plot, with central node
14 plt.axis([0,N,0,N])
15 plt.axes().set_aspect('equal')
16 for iteration in range(1,n_iter):           # Iteration loop
17
18     update=np.zeros([N,N],dtype='int')      # Set/reset evolution array
19
20     for i in range(1,N-1):                  # Lattice loops
21         for j in range(1,N-1):
22             cumul=0
23             for k in range(0,n_neighbor):   # Loop over nearest-neighbor
24                 cumul+=image[i+dx[k],j+dy[k]]
25             if image[i,j]==0 and cumul==1:  # Only one active neighbor
26                 update[i,j]=1               # Activate node
27                 plt.scatter(j+(i-N//2)/2.,N//2+0.866*(i-N//2)) # Plot node
28     # End of lattice loops
29
30     image+=update                           # Synchronous update of CA
31
32 # End of iteration loop
33 plt.show()                                  # Display structure
34 # END
```

Figure 2.6. Source code in the Python programming language for a 2-D CA defined over a triangular lattice with 6-neighbor connectivity. This is a minimal implementation, emphasizing conceptual clarity over programming elegance, code length, or run-time speed. The various matplotlib instructions (lines 13–15, 27, and 33) display the final structure, with activated nodes shifted horizontally and compressed vertically (line 27) so as to correctly display the structure in physical space, as in figure 2.7.

`image` updated (line 30). This *synchronous update* is necessary, otherwise the lattice update would depend on the order in which nodes are tested in the first set of lattice loops, thus introducing an undesirable spatial bias that would distort growth.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

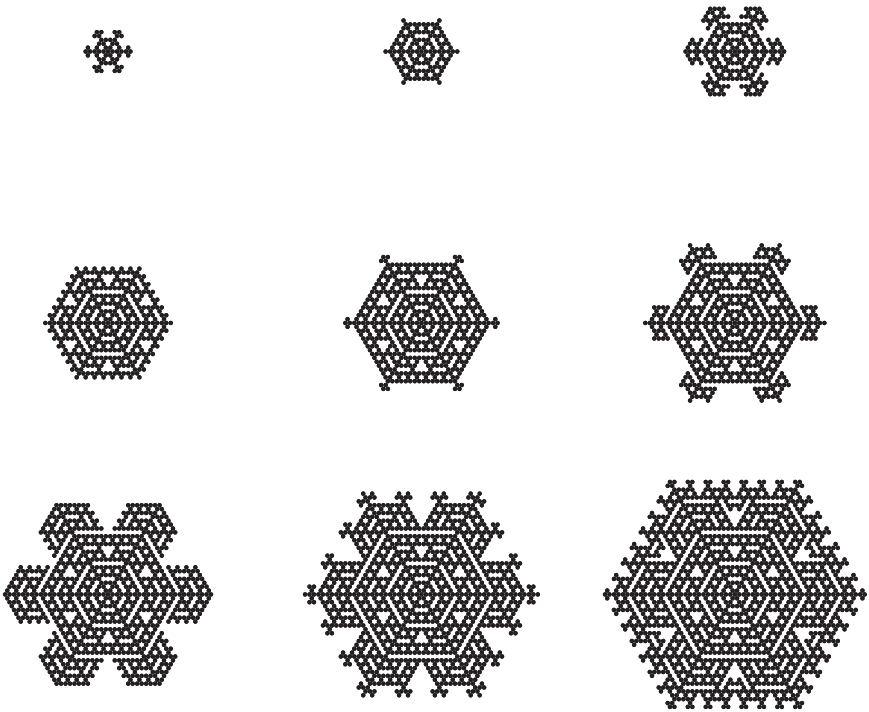

Figure 2.7. Structure growth generated by the 2-D CA with the simple one-neighbor rule on a triangular, 6-neighbor lattice. The top-left image shows the structure after 5 iterations, and the other images display the subsequent evolution at a 3-iteration cadence, the growth sequence being obvious. In the notation of section 2.3, this rule is written as (1) + 1.

Now consider growth beginning from a single occupied node (the "seed") at the center of a triangular lattice. The first three steps of the iterated growth process are illustrated via the color coding of lattice nodes in figure 2.5D. Starting from a single active node, the next iteration is a hexagonal ring of 6 active nodes (in red) surrounding the original active node (in black). At the next iteration, only the 6 nodes colored in orange abide by the

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

one-neighbor-only activation rule, but at the following iteration, each of these 6 "branches" will generate an arc-shaped clump of 3 active nodes (in yellow) at its extremity. Figure 2.7 picks up the growth at iteration 5 (top left), with subsequent frames plotted at a cadence of 3 iterations. As the six "spines" grow radially outward, the faces of the growing structure eventually fill inward from the corners, until a hexagonal shape is produced; from that point on, growth can pick up again only at the six corners, and later toward the centers of the faces, eventually adding another "layer" to the growing structure. The broken, concentric white hexagons within the structure, plotted in the bottom-right corner of figure 2.7, are the imprint of this layered growth process. Evidently, here the sixfold symmetry of the connectivity remains reflected in the global, "macroscopic" shape of the growing structure; this could perhaps have been expected, but certainly not the intricacies of details produced within the structure itself. In fact there is much more to these details than meets the eye; step back a bit to view the bottom-right structure from a distance, and it will hardly be distinguishable from the middle-left structure viewed at normal reading distance. This is again an indication of self-similarity.

All this being said, looking at figure 2.7, the first thing that comes to mind is, of course, snowflakes! It might appear ludicrous to suggest that our very artificial computational setup—triangular lattice, neighbor-based growth rule, etc.—has anything to do with the "natural" growth of snowflakes, but we will have occasion to revisit this issue in due time.

A similar 2-D CA simulation can be run on a Cartesian lattice with 8-neighbor connectivity, starting again with a single active node at the lattice center. All that is needed is to append two elements to the stencil arrays `dx` and `dy` in the code listed in figure 2.6. The first four steps of the growth process are again indicated by the nodal color coding in figure 2.5B. The first iteration produces a 3×3 block of active nodes, but at the

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

next iteration our one-neighbor rule makes growth possible only along diagonals quartering this 3×3 block (orange nodes). The next iteration (yellow nodes) generates a 5-node 90-degree wedge about each of the four extrusions generated at the preceding iteration; except for the fourfold symmetry, this is essentially the same growth pattern observed in sixfold symmetry on the triangular lattice (figure 2.5D). Figure 2.8 picks up the growth process at iteration 5, and subsequent frames are plotted at a 3-iteration cadence, as in figure 2.7. Growth now proceeds from the corners of the squares, which spawn more squares at their corners, and so on as the structure keeps growing, once again in a self-similar fashion.

2.3 A Zoo of 2-D Structures from Simple Rules

We henceforth restrict ourselves to a Cartesian lattice with 8-neighbor connectivity, and introduce a generalized 8-neighbor activation rule as follows: a node becomes active if either n_1 or n_2 neighboring nodes are active. We also include in the rule the number s of "seed" active nodes used to initialize the growth process. We write all this as

$$(n_1, n_2) + s, \quad 1 \leq n_1, n_2 \leq 8, \quad n_1 < n_2. \tag{2.1}$$

A specific example will likely help more than further explanation: The rule $(1, 5) + 1$ means that we start from one active node ($s = 1$). A node becomes active if either 1 or 5 neighboring nodes are active, and remains inactive otherwise, namely, if it has either 0, 2, 3, 4, 6, 7, or 8 active neighbors. Once again, no directional bias is introduced, as it doesn't matter where the 5 active nodes (say) are located in the 8-node group of neighboring nodes. Under this notation, the rules used to grow the structure in figure 2.7 would be written as $(1) + 1$.

Figure 2.9 shows a sample of structures grown using various such rules, as labeled. The variety of structures produced even by this narrow subset of rules is quite staggering, including again

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

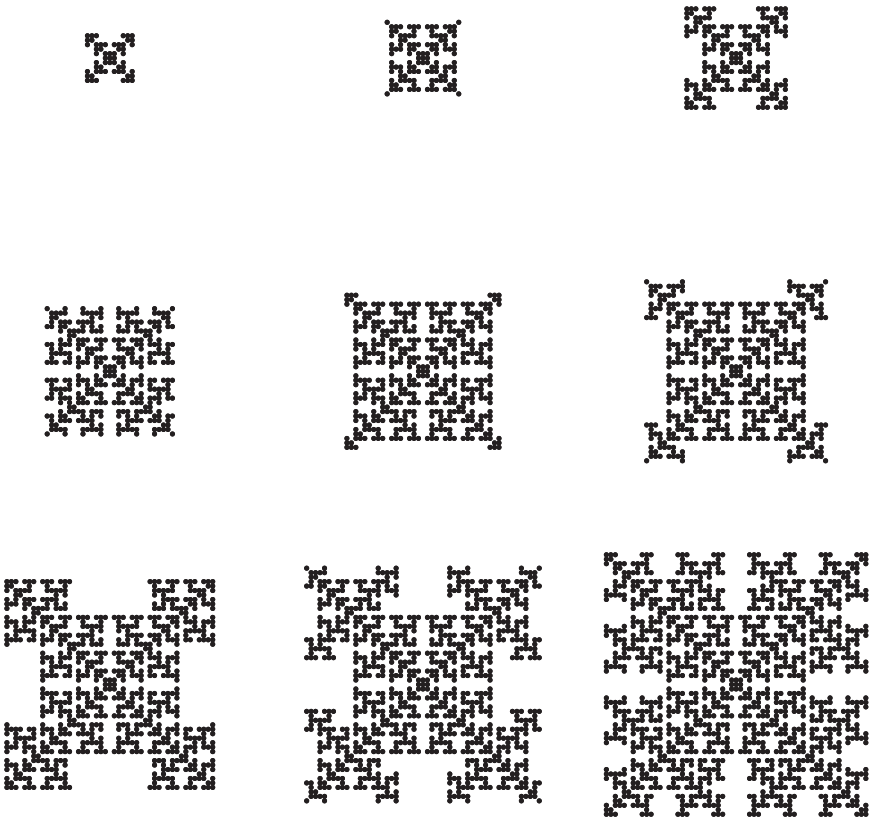

Figure 2.8. Structure generated by the 2-D CA with the simple rule (1) + 1, now on a Cartesian 8-neighbor lattice. The top-left image shows the structure after 5 iterations, and the other images display the subsequent evolution at a 3-iteration cadence, the growth sequence again being obvious.

self-similarity, a mixture of order and disorder, compact structures porous or solid, etc. Some rules, such as (3, 6) + 5, do not even generate regular outward growth, as extrusions fold back inward to fill deep crevices left open in earlier phases of the iterated growth process.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

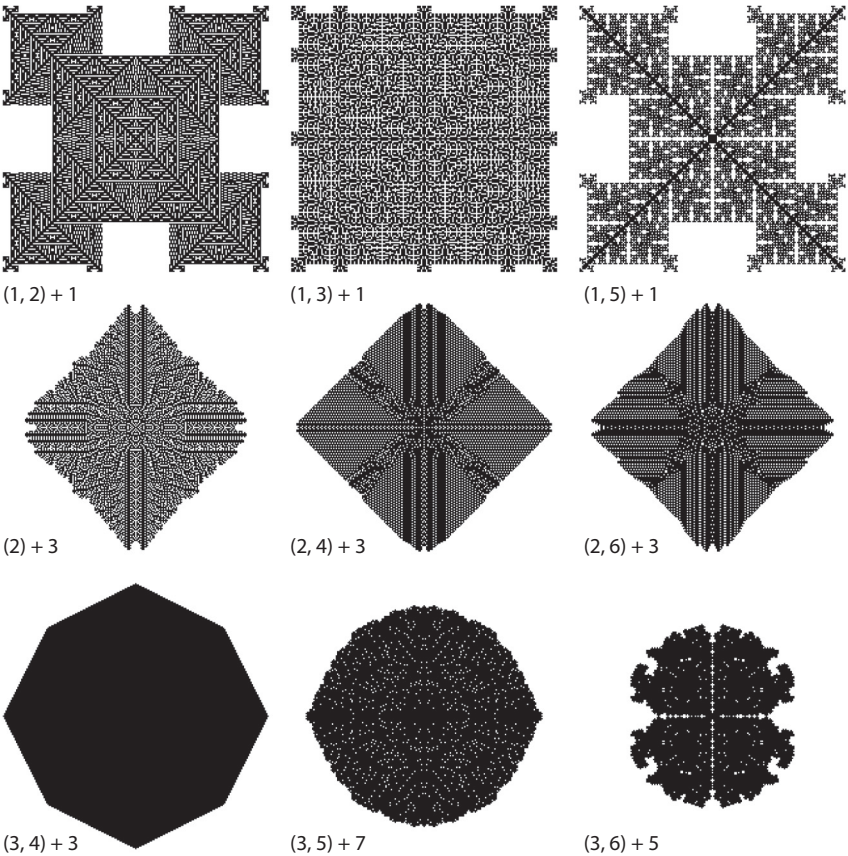

Figure 2.9. A zoo of structures grown by the 2-D CA on a Cartesian 8-neighbor lattice operating under a variety of rules, as labeled. All automata were executed for 100 iterations, except for the bottom three, for which 200 iterations were executed.

Staring at these and other structures generated by other specific incarnations of the two-member rule (2.1), one is naturally tempted to extract some general trends. For example, rules beginning with "1" produce squares that grow by spawning more squares at their corners, in a manner qualitatively similar to the

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

basic $(1) + 1$ rule; the numerical choice for n_2 affects primarily the internal pattern. Rules beginning with a "2" on the other hand, produce diamond-shaped structures, with ordered and disordered internal regions, growing along their four approximately linear faces; the numerical choice for n_2 mostly affects the relative importance of ordered and disordered regions in the interior. Rules with a "3" produce compact structures, sometimes solid sometimes porous, with various patterns of symmetry about vertical, horizontal, or diagonal axes present at the global scale. Now, if you find this convincing on the basis of figure 2.9, try running a simulation for rule $(3, 7) + 5$ and reconsider your position!

The overall conclusion of our relatively limited explorations of 2-D CAs remains the same as with the 1-D CAs considered previously: simple, microscopic growth rules can produce macroscopic structures ranging from highly regular to highly "complex," and very, very few of these structures could have been anticipated knowing only the lattice geometry and the growth rules.

2.4 Agents, Ants, and Highways

In the "classical" CAs considered thus far, the active elements are the lattice nodes themselves, and so they are fixed in space by definition. Another mechanism for iterated growth involves active elements moving on and interacting with the lattice (and/or with each other), according once again to set rules. Henceforth, such active elements will be defined as "agents." For example, an agent known as an "ant" moves and operates on a lattice as follows, from one iteration to the next:

- Move forward.
- If standing on a white node, paint it black and turn right by 90 degrees.
- If standing on a black node, paint it a whiter shade of pale (white) and turn left by 90 degrees.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

```
# HIGHWAY BUILDING BY LANGTON'S ANT
import numpy as np
import matplotlib.pyplot as plt
#-----------------------------------------------------------------------
N     =300                          # Lattice size
n_iter=20000                        # Number of temporal iterations
#-----------------------------------------------------------------------
x_step=np.array([0,-1,0,1])         # Template arrays for steps
y_step=np.array([1,0,-1,0])
image=np.zeros([N,N],dtype='int')   # Initialize lattice array, all white
ix=N//4                             # Ant's starting position in x
iy=N//4                             # Ant's starting position in y
direction=1                         # Ant's starting direction, North

for iteration in range(0,n_iter):   # Temporal loop

    ix+=x_step[direction]           # Ant moves
    iy+=y_step[direction]
    ix=(N+ix) % N                   # Enforce periodicity in x
    iy=(N+iy) % N                   # Enforce periodicity in y

    if image[iy,ix] == 0:           # On a white node
        update=1                    # Paint it black...
        direction+=1                # ...and turn right...
        direction=direction % 4     # ...but stay within step array
    else:                           # On a black node
        update=-1                   # Paint it white...
        direction-=1                # ...and turn left...
        direction=(4+direction) % 4 # ...but stay within step array

    image[iy,ix]+=update
# End of temporal loop
plt.imshow(image,interpolation="nearest")
plt.show()                          # Display final structure
# END
```

Figure 2.10. Source code in the Python programming language for an "ant" agent abiding by the rules introduced in section 2.4. This code generates the simulation plotted in the top panel of figure 2.11 (minus the colored dots and insets).

These are pretty simple behavioral rules, yet they hold surprises in store for us.

Figure 2.10 gives a simple numerical implementation of these behavioral rules. As with most codes listed throughout this book, logical clarity and readability have been given precedence over programming elegance, code length, or run-time speed, and computing-language-specific capabilities are systematically

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

avoided. Note the following:

1. The code is again structured as an outer temporal loop running over a preset number of temporal iterations `n_iter` (starting on line 15).
2. The two arrays `x_step` and `y_step` store the x- and y-increments associated with the four possible displacements, in the order down, left, up, right (lines 8–9). These are used to update the ant's position on the lattice (`ix, iy`) as per the ant's direction, stored in the variable `direction`. Under this ordering convention, turning right requires incrementing `direction` by $+1$ (line 24), and turning left by -1 (line 28).
3. The modulus operator % is used to enforce periodicity for the ant's position (lines 19–20) and stepping direction (lines 25 and 29). The instruction $a\%b$ returns the remainder of the division of a by b, for example $7\%3 = 1$, $2\%3 = 2$, $3\%3 = 0$. See appendix A.3 for more on the use of the modulus operator in Python.
4. The *change* in the lattice state at the ant's position is first calculated (variable `update`) and the lattice is updated (line 31) only once the **if**. . . **else** construct is exited. This is needed because the lattice state at the ant's position sets the operating condition of this logical structure, so changing its value *within* its blocks of instructions is definitely not a good idea in most programming languages.

The top panel in figure 2.11 shows the structure built by a single ant moving on a 300×300 lattice, starting at the location marked by the red dot, and initially pointing north (top of the page). The initial state of the lattice is all white nodes. These are the parameter settings and initial conditions implemented in

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

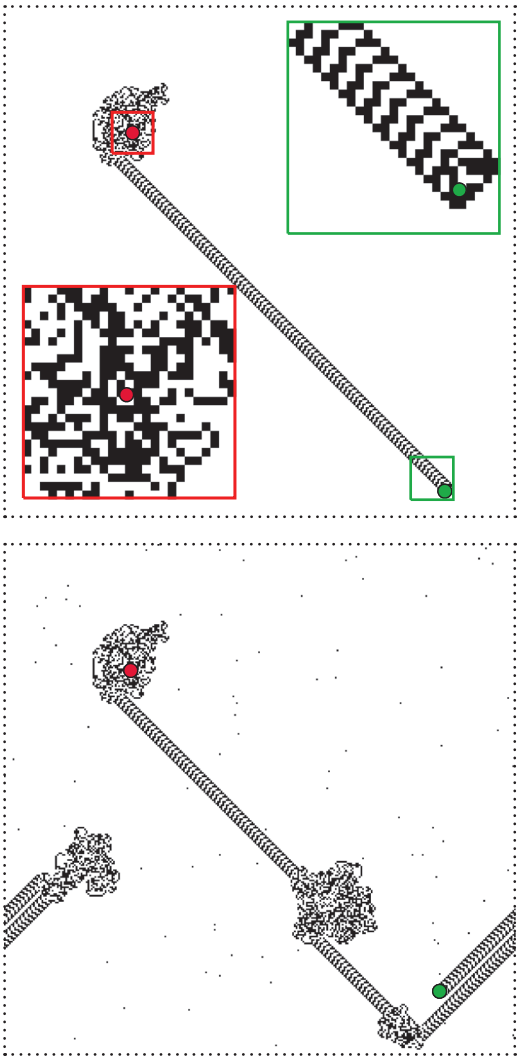

Figure 2.11. Highway building by an "ant agent" in a clean (top; 20,000 iterations) and noisy (bottom; 51,000 iterations) background environment. The solid dots show the starting (red) and ending (green) position of the ant, with the insets in the top panel providing close-ups of the lattice about these two points. The lattice is assumed periodic both horizontally and vertically. See appendix D.2 for more on periodic boundary conditions on lattices.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

the code listed in figure 2.10. The first few thousands iterations, shown in the inset framed in red, produce, if not strictly random, at least a highly disordered clump of white and black nodes. But after a bit more than 10,000 time steps, a switch to a different behavior takes place. The ant now executes a periodic series of steps, involving a lot of backtracking but also a net drift velocity along a diagonal with respect to the lines of the Cartesian lattice, leaving behind in its trail a highly ordered, spatially periodic pattern of white and black nodes (see green inset). This behavior has been dubbed "highway building," and it could hardly have been expected on the basis of the ant's simple behavioral rules. Highway building always proceeds along 45-degree diagonals, and once started would go on forever on an infinite-sized lattice. The fact that the highway points to the southeast in figure 2.11 is determined by the initial conditions: all white nodes and the ant pointing north.

In practice, simulations such as in figure 2.11 are carried out on a finite-sized lattice, on which horizontal and vertical periodicity is enforced. So here, pushing the simulation further would eventually lead to the ant (and its highway) leaving the lattice near the southeast corner, to reappear near the northwest corner, still heading southeast, eventually hitting the structure it just built. This throws the ant into a fit; disordered (re)painting prevails for a while, forming a structure statistically similar to that characterizing the first 10^4 iterations, but after many thousands of iterations highway building resumes, in a direction orthogonal to that of the original highway, to the southwest here. As the lattice fills with blotches of disorder and stretches of highway crossing each other, highway building becomes increasingly difficult, and if the evolution is pushed sufficiently far, on any finite-sized lattice the end result is randomness.

Highway building is a pretty delicate process that is easily disturbed. The bottom panel of figure 2.11 shows what happens when a small number of randomly selected lattice nodes are

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

painted black before the ant starts moving. At first, the evolution proceeds as before, and highway building toward the southeast begins, but soon the ant hits one of the randomly distributed black nodes, triggering disordered painting. Highway building eventually resumes, still toward the southeast, until another black node is encountered, triggering another, shorter, disordered episode that ends with highway building resuming now toward the northeast; and so on over the 51,000 iterations over which this specific simulation was pursued.

2.5 Emergent Structures and Behaviors

We have barely touched the realm of structures and behaviors that can be produced by CAs and CA-like systems. Nonetheless, the take-home message of this chapter should be already clear: very simple rules can produce very complex-looking structures. But should we really be calling these structures "complex" if their generating rules are simple? Or do we remain tied to an intuitive definition of "complex" relying on our visual perception of structures and behaviors? Students of complexity have been rattling their brains over that one for quite a while now.

If defining complexity is hard, coming up with an unambiguous and universal quantitative *measure* of complexity is perhaps the next hardest thing. Many such measures have been defined, and can be categorized in terms of the question they attempt to answer; given a complex structure or system, the following three queries are all pertinent:

1. How hard is it to describe?
2. How hard is it to create?
3. How is it dynamically organized?

Consider, for example, the measure known as *algorithmic complexity*, namely, the length of the smallest computer program that can generate a given output—a spatial pattern, a time series, a network, whatever. It may appear eminently reasonable to

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

suppose that more complex patterns require longer programs; simulating the evolving climate certainly requires much longer code (and a lot more computer time!) than simulating the harmonic oscillation of a frictionless pendulum. It seems to make sense, but we need to look no further than the simple 1-D CAs investigated in section 2.1 to realize the limitations of this measure of complexity. The CAs of figures 2.1, 2.2, and 2.3 can be produced by programs of exactly the same length, yet they could hardly be considered "equally complex."

Our brief foray into cellular automata also highlights a theme that will recur throughout this book and that is almost universally considered a defining feature of natural complexity: *emergence*. This term is used to refer to the fact that global structures or behaviors on macroscopic scales cannot be reduced to (or inferred from) the rules operating at the microscopic level of individual components making up the system; instead, they emerge from the *interactions* between these components. Synthetic snowflakes and ant highways are such examples of emergence, and are by no means the last to be encountered in this book.

2.6 Exercises and Further Computational Explorations

1. Use the 1-D CA code of figure 2.4 to explore the behavior of the four CA rules introduced in section 2.1 when starting from a random initial condition, i.e., each cell is randomly assigned to be black or white with equal probability. If needed, see appendix C.2 for a quick start on generating uniformly distributed random deviates in Python. Make sure also to enforce periodic boundary conditions (see appendix D.2).
2. The aim of this exercise is to explore further the patterns produced by 1-D CA rules, all of which are relatively easy to implement in the source code of figure 2.4. The following five individual rules produce patterns qualitatively distinct from those already

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

examined in section 2.1. Unless the pattern looks really trivial, make sure to run the CA for enough iterations to ascertain its long-term behavior.

- Cell j becomes (or stays) black only if both $j-1$ and j are white; otherwise the cell stays (or becomes) white.
- Cell j becomes (or stays) black if any two cells of the triad $[j-1, j, j+1]$ are black, or if both $j-1$ and j are white; otherwise the cell stays (or becomes) white.
- Cell j becomes (or stays) white if $j-1$ and j are both black, or if the triad $[j-1, j, j+1]$ are all white; otherwise the cell stays (or becomes) black.
- Cell j becomes (or stays) black if cell $j-1$ and at least one of the pair $[j, j+1]$ are black, or if the triad $[j-1, j, j+1]$ are all white; otherwise the cell stays (or becomes) white.
- Cell j becomes (or stays) white if $j+1$ and j are both white, or if the triad $[j-1, j, j+1]$ are all black; otherwise the cell stays (or becomes) black.

The last two rules, in particular, should be iterated over many hundreds of iterations over a large lattice; the patterns produced are particularly intriguing. You should also run these five rules starting from a random initial condition. To which CA class does each belong?

3. Modify the code of figure 2.6 to introduce two-member rules such as those used to produce figure 2.9 on the 6-neighbor triangular lattice. Explore the growth produced by the set of rules $(1, 2) + 1$ through

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

$(1, 6) + 1$. Is lattice structure always imprinted on global shape?

4. Modify the code of figure 2.6 to operate on a Cartesian 8-neighbor lattice, and explore the sensitivity to initial conditions for rules $(3, 4) + n$ and $(3, 5) + n$. More specifically, consider the growth produced by using either $n = 3$, 4, or 5 active nodes, organized either linearly, as a 2×2 block, as a diamond-shaped 5-node block, etc. Is the geometry of the initial condition imprinted on the global shape?
5. The Game of Life is one of the most intensely studied 2-D CAs. It is defined on a 2-D Cartesian lattice, periodic horizontally and vertically, with 8-neighbor connectivity. Each lattice node can be in one of two possible states, say "inactive" and "active" (or 0 and 1; or white and black; or dead and alive; whatever), and evolves from one iteration to the next according to the following rules:

   - If an active node has less than two active neighbors, it becomes inactive.
   - If an active node has more than three active neighbors, it becomes inactive.
   - If an inactive node has three active neighbors, it becomes active.
   - If a node has two active neighbors, it remains in its current state.

   This automaton can generate "organisms," i.e., shape-preserving structures moving on the lattice, in some cases interacting with one another or with their environment to produce even more intricate behaviors. Modify the code of figure 2.6 to incorporate the above rules, and run simulations starting from a random initialization of the lattice in which each node is

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

assigned active or inactive status with equal probability.

6. The Grand Challenge for this chapter is to explore the behavior of another interesting ant-like agent, known as the "termite." Termites move randomly on a lattice on which "wood chips" (i.e., black) have been randomly dispersed. The termite's behavioral rules are the following:

   - Random walk until a wood chip is encountered.
   - **if** a wood chip is currently being carried, drop it at the current position (i.e., next to the one just bumped into), and resume the random walk.
   - **else** pick up the chip bumped into, and resume the random walk.

   Code this up, perhaps starting from the "ant" code of figure 2.10. Appendix D.3 may prove useful if you need a kick-start on how to code up a random walk on a lattice. How is the distribution of wood chips evolving with time? Does this change if you let more than one termite loose on the lattice?

2.7 Further Reading

Pretty much anything and everything that could be written on CAs can be found in

Wolfram, S., *A New Kind of Science*, Wolfram Media (2002).

The material covered in sections 2.1 and 2.2 follows rather closely parts of chapters 2 and 8 of this massive tome. The Wikipedia page on CAs includes a good discussion of the history of CA research, with copious references to the early literature (viewed March 2015).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

For a succinct and engaging introduction to virtual ants and similar computational insects, see

Resnick, M., *Turtles, Termites, and Traffic Jams*, MIT Press (1994);

as well as chapter 16 in

Flake, G.W., *The Computational Beauty of Nature*, MIT Press (1998).

Chapter 15 of this volume also offers a nice introduction to CAs, including the Game of Life. As far as I know, the general notion of an "agent" has been borrowed from economics and introduced into complexity science by John Holland; for more on this concept see

Holland, J.H., *Hidden Order*, Reading: Addison-Wesley (1995).

Some years ago the term *artificial life* was coined to define a category for computational ants, termites, turmites, boids, and other similarly designed computational critters, as well as those appearing in John Conway's Game of Life. The following collection of papers remains a great overview of this computational zoology and its underlying motivations:

Langton, C., ed., *Artificial Life*, MIT Press (1995).

Langton is actually the designer of the ant agent starring in section 2.4. The Wikipedia page on Langton's ant is worth viewing; it also provides examples of extensions to multiple-state ants, as well as a good sample of references to the technical literature:

http://en.wikipedia.org/wiki/Langton%27s_ant (viewed March 2015),

and, not to be missed, a detailed study of the real thing:

Gordon, D.M., *Ant Encounters: Interaction Networks and Colony Behavior*, Princeton University Press (2010).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms

On measuring complexity, see

Lloyd, S., "Measures of complexity: A nonexhaustive list," *IEEE Control Syst. Mag.*, **21** (2001);

Machta, J., "Natural complexity, computational complexity and depth," *Chaos*, **21**, 037111 (2011).

The three-questions categorization of complexity measures introduced in section 2.5 is taken from the first of these papers, which lists no less than 42 such measures, in a "nonexhaustive list"! The second paper is part of a focus issue of the research journal *Chaos*, devoted to measures of complexity.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:45:40 UTC
All use subject to https://about.jstor.org/terms